

Documentation

1

The Importance of Documentation

- Effective documentation is **crucial** for software development projects.
- **Clear, concise, and user-friendly** documentation contributes significantly to the success of a codebase.
- Many developers abandon code due to **lack of documentation**, making it difficult to get started.



2

Types of Technical Documentation

- **Plain text, images, and videos** explaining core software concepts
- **API documentation** outlining function/class calls, parameters, etc.
- **Code examples** demonstrating functionality usage
- **Many other formats** exist beyond these.



Documentation Systems

- Numerous documentation systems are available:
 - MkDocs
 - Sphinx
 - GitBook
 - Docusaurus
 - Doxygen
 - Jekyll
- These are **static site generators**: the HTML code remains unchanged once generated and served, even with dynamic elements like videos.



MkDocs

- MkDocs is a documentation system that uses **markdown** for writing documentation and **Python** for the build system.
- The “**material for mkdocs theme**” enhances MkDocs with customization options for professional static sites.
- This course utilizes MkDocs with the material theme.



Mkdocs Configuration File

- The **mkdocs.yaml** file is the core configuration file for MkDocs projects.
- It defines:
 - **Project name and author**
 - **Source directory** for documentation files
 - **Theme** (material recommended)
 - **Features** enabled by the theme (e.g., code copy, annotations)
 - **Plugins** for added functionality (e.g., search, docstrings)
 - **Navigation structure** of the documentation



Documenting Your API

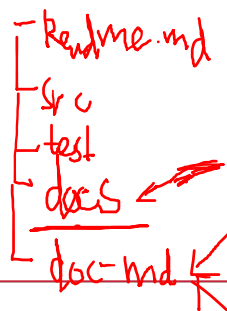
- Ensure your `src` module contains at least one function and class.
- Create a markdown file (e.g., `my_api.md`) in the `docs/source` folder.
- Add the file to the `nav` section in `mkddocs.yaml`.
- Use the `:::` indicator to specify function/module paths for rendering.



7

Building and Publishing Documentation

- Build the documentation using `mkddocs build` to generate a site folder containing HTML code.
- Host the built documentation using services like **GitHub Pages**.
- GitHub Pages is free for public projects.
- Utilize **GitHub Actions workflow files** to automate the deployment process.



8

Importance of Documentation for Project Maintenance

- Writing **proper documentation** is **essential** for larger projects that require long-term maintenance.
- Documentation is an iterative process best done alongside code writing.



Exercise

- We are going to need two Python packages to get started: [mkdocs](#) and [material for mkdocs](#). Install with

```
pip install "mkdocs-material >= 4.8.0" # (1)!
```

- Run in your terminal (from the docs folder):

```
mkdocs serve # (1)!
```



A source file

```
src/models/model.py file

import torch

class MyNeuralNet(torch.nn.Module):
    """Basic neural network class.

    Args:
        in_features: number of input features
        out_features: number of output features
    """
    def __init__(self, in_features: int, out_features: int) -> None:
        self.l1 = torch.nn.Linear(in_features, 500)
        self.l2 = torch.nn.Linear(500, out_features)
        self.r = torch.nn.ReLU()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Forward pass of the model.

        Args:
            x: input tensor expected to be of shape [N,in_features]

        Returns:
            Output tensor with shape [N,out_features]
        """
        return self.l2(self.r(self.l1(x)))

and the following function to add src/predict_model.py file:

def predict(
    model: torch.nn.Module,
    dataloader: torch.utils.data.DataLoader
) -> None:
    """Run prediction for a given model and dataloader.

    Args:
        model: model to use for prediction
        dataloader: dataloader with batches

    Returns:
        Tensor of shape [N, d] where N is the number of samples and d is the output dimension of the model
    """
    return [model(batch) for batch in dataloader]
```



11

Add Markdown files

- Add a markdown file to the docs/source folder called `my_api.md` and add that file to the `nav:` section in the `mkdocs.yml` file.
- To that file add the following code:

```
# My API

::: src.models.model.MyNeuralNet
::: src.predict_model.predict
```

The `:::` indicator tells mkdocs that it should look for the corresponding function/module and then render it on the given page. Thus, if you have a function/module located in another location change the paths accordingly.

- Make sure that the documentation correctly includes your function and module on the given page.
- Finally, try to build a final version of your documentation
 - `mkdocs build`



12